

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 1

Design Desk

COURSE NAME: Deep Learning

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO5: Students will be able to analyze and explain advanced generative deep learning models including Autoencoders, Deep Belief Networks, Boltzmann Machines, Deep Boltzmann Machines, and Generative Adversarial Networks. They will be able to compare their architectures, explain their learning mechanisms, and apply suitable generative models to real-world problems. (BTL-3)

Assessment Tool Weightage: 30%

QUESTIONS:

1. Design an Autoencoder-Based Feature Learning System

A manufacturing company wants to detect defective products from high-dimensional sensor data. Design an Undercomplete Autoencoder with suitable encoder, latent representation, decoder, activation functions, and reconstruction loss. Explain how the design performs dimensionality reduction and feature learning.

2. Design a Robust Regularized Autoencoder

A healthcare organization has noisy patient-record features and needs a robust representation for downstream classification. Design a Regularized Autoencoder using an appropriate regularization strategy. Explain the encoder-decoder structure, regularization mechanism, training objective, and how the design reduces overfitting and improves useful feature extraction.

3. Design a Deep Generative Model

A recommendation or content-generation system requires learning complex data distributions. Design a Deep Belief Network or Deep Boltzmann Machine using suitable visible and hidden layers. Illustrate the architecture, explain the learning process, and justify the choice of the model for the proposed application.

4. Design a Stochastic and Contractive Representation Model

A real-world image or signal-processing application requires representations that remain useful under small input variations. Design a model using Stochastic Encoders/Decoders or a Contractive Encoder. Specify the architecture, objective function, and robustness mechanism, and explain how the design improves representation stability.

5. Design a GAN-Based Data Generation System

An organization has limited training images and wants to generate realistic synthetic samples. Design a Generative Adversarial Network with a Generator and Discriminator. Specify the input, network flow, adversarial training process, loss functions, and evaluation approach. Compare the proposed GAN design with an Autoencoder and justify why GAN is suitable for the application.

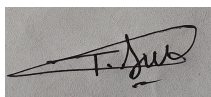
Design Desk Guidelines

- Identify the real-world problem, requirements, inputs, outputs and constraints.
- Draw a clear architecture or workflow diagram for the proposed model.
- Specify important layers, units, activation functions, latent representation and loss functions.
- Explain the training or learning mechanism and justify major design choices.
- Mention suitable evaluation measures and expected outcomes.
- Use neat labeling and present the design in a logical sequence.
- The solution should be original and written in the student's own words.

Code of Conduct:

I T Narasimha(192425233)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Understanding of Problem Context	20	Clearly understands the problem, requirements, constraints, and expected outcomes.	Good understanding with minor gaps in requirements.	Basic understanding; some requirements are unclear.	Poor understanding or misinterpretation of the problem.
Application of Concepts	30	Accurately applies appropriate generative deep learning concepts and architectures.	Applies relevant concepts correctly with minor errors.	Applies basic concepts with limited accuracy.	Fails to apply appropriate concepts.
Design Approach	20	Innovative, logical, well-structured, feasible, and technically justified design.	Logical and feasible design with minor gaps.	Basic design with some inconsistencies.	Unclear, illogical, or impractical design.
Accuracy of Solution	20	Completely correct architecture, process, objectives, and justification.	Mostly correct with minor technical errors.	Partially correct solution with noticeable gaps.	Incorrect or incomplete solution.
Clarity	10	Clear, well-organized diagram and explanation with proper technical labeling.	Understandable with minor clarity issues.	Limited clarity and explanation.	Poorly presented and difficult to understand.

SIMATS ENGINEERING
CO5 - ASSESSMENT TOOL 1 (Design Desk)
Course: Deep Learning (MLA0408)

1. Design an Autoencoder-Based Feature Learning System

Problem Context

A manufacturing company collects high-dimensional sensor readings (vibration, temperature, current, acoustic emission, pressure) from production-line machinery. Manually inspecting every dimension for anomalies is expensive, and the defect class is rare, so a supervised classifier alone would suffer from severe class imbalance. The requirement is a model that learns a compact representation of "normal" sensor behaviour and flags a product as defective when its sensor signature reconstructs poorly, i.e. it deviates from the learned normal manifold.

Architecture

An Undercomplete Autoencoder is proposed because its bottleneck forces the network to retain only the most informative, non-redundant patterns of normal sensor behaviour, which is exactly the dimensionality-reduction and feature-learning capability the problem needs.

- Input layer: n = number of raw sensor features (e.g. 200 dimensions after basic pre-processing/normalisation).
- Encoder: fully connected layers $200 \rightarrow 128 \rightarrow 64 \rightarrow 32$, each followed by a ReLU activation, progressively compressing the input.
- Latent (bottleneck) layer: 16 units, dimensionality far lower than the input, forcing the network to learn only the dominant, non-redundant factors of variation in normal operating data.
- Decoder: mirror of the encoder, $16 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 200$, with ReLU activations on hidden layers and a linear (or sigmoid, if inputs are scaled to $[0,1]$) activation on the output layer so reconstructed values can match the real-valued sensor range.

Loss Function and Training

The network is trained only on sensor data collected from non-defective products, using Mean Squared Error between the input vector x and its reconstruction $\hat{x}$ as the reconstruction loss: $L(x, \hat{x}) = (1/n) * \sum((x_i - \hat{x}_i)^2)$. Training uses the Adam optimiser with a small learning rate (around $1e-3$), mini-batches, and early stopping on a held-out validation set of normal samples to prevent the network from overfitting to noise.

How the Design Performs Dimensionality Reduction and Defect Detection

Because the bottleneck (16 units) is much smaller than the input (200 units), the encoder cannot simply copy the input forward; it must discover a lower-dimensional latent code that captures the

correlated structure shared by normal sensor readings, which is precisely dimensionality reduction and unsupervised feature learning. At inference time, a new product's sensor vector is passed through the trained autoencoder and its reconstruction error is computed. Since the network was never exposed to defective patterns, defective sensor signatures fall outside the learned manifold and reconstruct poorly, producing a high error. A threshold on the reconstruction error (e.g. set using the 95th/99th percentile of validation errors on normal data) is then used to flag a product as defective.

Evaluation and Expected Outcome

Model quality is evaluated using the reconstruction-error distribution separability between normal and defective samples, and, once a threshold is chosen, standard classification metrics such as precision, recall, F1-score and ROC-AUC on a labelled test set. The expected outcome is a compact 16-dimensional latent representation that captures the intrinsic structure of normal sensor behaviour and a reconstruction-error-based rule that reliably separates defective products from normal ones without requiring large amounts of labelled defect data.

2. Design a Robust Regularized Autoencoder

Problem Context

A healthcare organisation has patient-record features (vital signs, lab values, categorical indicators) that are noisy due to sensor error, missing entries and human data-entry mistakes. The organisation needs a robust feature representation that a downstream classifier (e.g. for risk prediction) can use, so the representation must not simply memorise noise but must generalise to the true underlying patient state.

Architecture

A Denoising Autoencoder (a regularised autoencoder that uses noise injection as its regularisation strategy) is proposed.

- Encoder: input dimension $d \rightarrow 128 \rightarrow 64 \rightarrow$ latent dimension 20, ReLU activations at each hidden layer.
- Latent layer: 20 units representing the denoised, compact patient feature embedding.
- Decoder: $20 \rightarrow 64 \rightarrow 128 \rightarrow d$, ReLU on hidden layers and a linear/sigmoid output layer depending on feature scaling.

Regularisation Mechanism and Training Objective

During training, controlled corruption is applied to each input record x to produce a corrupted version $\tilde{x}$, using Gaussian noise on continuous lab values and random masking (dropout-style zeroing) on categorical/binary fields. The encoder-decoder is trained to reconstruct the original clean record from the corrupted one, giving the objective $L = \|x - g(f(\tilde{x}))\|^2$, where f is the

encoder and g is the decoder. Because the network must recover clean values from corrupted inputs, it cannot rely on the identity mapping and is forced to learn the true statistical dependencies between patient features rather than memorising exact input values, which is a strong form of regularisation.

How This Reduces Overfitting and Improves Feature Extraction

Denoising regularisation prevents the well-known failure mode of plain autoencoders on noisy or small clinical datasets, where the network could otherwise overfit and learn a near-identity mapping that reproduces noise along with signal. By repeatedly seeing many corrupted views of the same underlying patient record, the model learns a representation that is stable under small perturbations, effectively capturing the manifold of plausible clean patient states. This yields latent features that are more informative and less noise-sensitive for the downstream classifier, improving generalisation on unseen patients.

Evaluation and Expected Outcome

The autoencoder is evaluated using reconstruction error on a held-out clean validation set, and, more importantly, by the downstream classification performance (accuracy, F1-score, AUC) obtained when the learned 20-dimensional embeddings replace or augment the raw noisy features. The expected outcome is a compact, denoised patient representation that improves downstream classification robustness compared to using raw noisy features directly.

3. Design a Deep Generative Model

Problem Context

A recommendation/content-generation system must learn the complex, high-dimensional joint distribution over user-item interactions or content features so that it can generate plausible new recommendations or content patterns rather than merely memorising the training data. This requires a model capable of learning multi-layered, hierarchical feature representations of the underlying data distribution.

Chosen Model and Architecture

A Deep Belief Network (DBN) is proposed, built by stacking multiple Restricted Boltzmann Machines (RBMs).

- Visible layer v : encodes the raw input, e.g. binary/normalised user-item interaction vectors or content feature vectors.
- Hidden layer 1 (h_1): 256 units, connected to v through an RBM; captures first-order co-occurrence patterns (e.g. item-item correlations).
- Hidden layer 2 (h_2): 128 units, connected to h_1 through a second RBM; captures higher-order combinations of the first-layer features (e.g. latent taste/topic groups).

- Hidden layer 3 (h3, top layer): 64 units, forming the deepest, most abstract representation of user preference or content style, on top of which an associative memory / classifier can be attached for recommendation scoring.

Learning Process

The DBN is trained using greedy layer-wise pre-training. Each RBM is trained independently and unsupervised using Contrastive Divergence: visible units are sampled, hidden activations are computed, a reconstruction is sampled back to the visible layer, and weights are updated to increase the probability the model assigns to the training data, following the update rule $\Delta(W)$ proportional to $(\langle v_i h_j \rangle_{\text{data}} - \langle v_i h_j \rangle_{\text{reconstruction}})$. Once the first RBM converges, its hidden-layer activations become the "visible" input for training the next RBM, and this is repeated layer by layer. After all layers are pre-trained, the whole stacked network can optionally be fine-tuned with a small amount of labelled feedback (e.g. click/purchase signals) using backpropagation, converting the generative DBN into a discriminative-generative hybrid suited for scoring recommendations.

Justification of the Model Choice

A DBN is well suited to this application because greedy layer-wise pre-training allows the network to learn a good hierarchical initialisation purely from abundant unlabelled interaction/content data, which is exactly the situation in recommendation systems where explicit labels (ratings, purchases) are sparse compared to raw interaction logs. The stacked RBM structure lets the model capture increasingly abstract taste or content patterns layer by layer, which supports both generating new plausible interaction patterns (content/recommendation generation) and providing a strong feature representation for a final supervised recommendation layer.

Evaluation and Expected Outcome

Evaluation combines generative quality measures, such as reconstruction/log-likelihood on held-out interaction data, with recommendation-specific metrics such as precision@k, recall@k and NDCG after fine-tuning. The expected outcome is a hierarchy of latent features that captures user taste or content style at multiple levels of abstraction, improving both the realism of generated recommendations and downstream ranking quality.

4. Design a Stochastic and Contractive Representation Model

Problem Context

An image or signal-processing application must produce representations that stay useful even when the input is affected by small variations such as sensor noise, minor lighting change, or slight signal jitter. A plain autoencoder can be sensitive to such perturbations, since nothing in its objective explicitly penalises sensitivity of the encoding to small input changes.

Chosen Model and Architecture

A Contractive Autoencoder (CAE) is proposed, which explicitly penalises the sensitivity of the learned representation to small input perturbations.

- Encoder: Convolutional layers (for image input) or fully connected layers (for generic signal input) mapping the input x to a latent code $h = f(x)$, with ReLU/sigmoid activations.
- Latent layer: a compact code h (e.g. 32-64 units for signals, or a small feature map for images) that must remain stable under small input changes.
- Decoder: mirrors the encoder to reconstruct $\hat{x} = g(h)$, producing the reconstructed image/signal.

Objective Function and Robustness Mechanism

In addition to the usual reconstruction loss, the CAE adds a contractive penalty equal to the squared Frobenius norm of the Jacobian of the encoder's hidden activations with respect to the input: $L =$

$\|x - g(f(x))\|^2 + \lambda * \|J_{f(x)}\|_F^2$, where $J_{f(x)}$ is the Jacobian of h with respect to x and λ controls the strength of the penalty. Minimising the Jacobian norm forces the encoder function to be flat (locally insensitive) in most input directions around each training point, while the reconstruction term still forces it to be sensitive enough along the directions that matter for reconstructing the data manifold. As an alternative or complementary strategy, a Stochastic Encoder/Decoder can be used, where the encoder outputs the parameters of a distribution (e.g. mean and variance) rather than a single deterministic code, and the code is sampled from this distribution during training, which similarly discourages the model from relying on exact, brittle input values.

How the Design Improves Representation Stability

Because the contractive penalty explicitly discourages the encoder from producing large output changes for small input changes, the learned latent code becomes locally invariant to noise and minor variations while still preserving the information needed to reconstruct the true signal or image content. This makes the representation more reliable for downstream tasks (classification, retrieval, anomaly detection) operating on real-world, imperfect sensor or image data, where robustness to small perturbations is essential.

Evaluation and Expected Outcome

The model is evaluated by comparing reconstruction error and downstream task accuracy on clean versus artificially perturbed test inputs; a well-trained CAE shows only a small performance drop under perturbation compared to a plain autoencoder. The expected outcome is a latent representation whose Jacobian sensitivity is low, giving stable, noise-robust features for the target image/signal application.

5. Design a GAN-Based Data Generation System

Problem Context

An organisation has a limited set of training images and needs additional realistic synthetic samples to augment its dataset, for example to improve a downstream classifier's generalisation. The requirement is a model that can learn the true data distribution of the available images well enough to sample new, realistic examples from it, rather than merely compressing existing images.

Architecture

- Generator G: takes a random noise vector z (e.g. 100-dimensional, sampled from a standard normal distribution) as input and passes it through a series of transposed-convolution (upsampling) layers with batch normalisation and ReLU activations, ending in a tanh output layer, to produce a synthetic image $G(z)$ of the target resolution.
- Discriminator D: takes an image (either a real training image or a generated image $G(z)$) and passes it through convolutional layers with LeakyReLU activations and batch normalisation, ending in a single sigmoid output unit that estimates the probability the input image is real.

Network Flow and Adversarial Training Process

Training alternates between two steps in each iteration. First, the discriminator is updated using a batch of real images (label 1) and a batch of generator-produced fake images (label 0), so it learns to distinguish real from fake. Second, the generator is updated by passing new noise vectors through G, then through the (now frozen) discriminator, and adjusting G's weights to make D classify the fakes as real, i.e. to push $D(G(z))$ towards 1. This adversarial process is a minimax game between the two networks, each improving in response to the other, and, when it converges, drives the generator's output distribution towards the true data distribution.

Loss Functions

The discriminator loss is the binary cross-entropy: $L_D = -E[\log D(x)] - E[\log(1 - D(G(z)))]$, maximised with respect to D's parameters. The generator loss uses the standard non-saturating form: $L_G = -E[\log D(G(z))]$, minimised with respect to G's parameters so that early-training gradients remain strong even when the discriminator is confidently rejecting obviously fake images.

Evaluation Approach

Generated samples are evaluated using a combination of qualitative visual inspection, quantitative distributional metrics such as Frechet Inception Distance (FID) and Inception Score, and, since the goal is data augmentation, by measuring the improvement in a downstream classifier's accuracy when trained on the original data combined with GAN-generated samples versus the original data alone.

Comparison with an Autoencoder and Justification for GAN

An autoencoder is trained to reconstruct its own input by minimising pixel-wise reconstruction error, which typically produces blurry outputs because the loss averages over plausible pixel values and the model is never explicitly pushed to produce sharp, realistic samples; it is also not naturally designed for generating brand-new samples from noise, since a plain autoencoder's latent space is not guaranteed to be smoothly sample-able without further constraints (as in a Variational Autoencoder). A GAN, in contrast, is explicitly trained through the adversarial signal from the discriminator to produce outputs that are indistinguishable from real data, which directly optimises for realism rather than pixel-wise averaging, and its generator is designed from the start to map random noise to new, diverse samples. For an organisation whose goal is to synthesise realistic new images from limited data, the GAN design is therefore more suitable than an autoencoder, since it directly targets sample realism and diversity rather than reconstruction fidelity of existing inputs.